Welcome to **Shell-Scripting**

Introduction to Shell Scripting

Shell scripting is writing a series of command lines in a file to be executed by the shell, automating tasks that could be done manually in the terminal. It combines commands, logic, and functions to automate complex tasks efficiently.

What is a Shell?

A shell is a command-line interpreter that provides a user interface for the Unix-like operating systems. It allows users to execute commands, run programs, and manage system resources. Shells can be interactive (executing commands as entered) or non-interactive (executing commands from a script).

Types of Shells

1. **Bourne Shell (sh) :** The original Unix shell.
2. **Bash Shell (bash) :** An enhanced version of the Bourne Shell, widely used due to its advanced features.
3. **Korn Shell (ksh) :** A shell developed by David Korn, combining features of both the Bourne and C shells.
4. **C Shell (csh) :** Known for its C-like syntax.
5. **Z Shell (zsh) :** An extended Bourne shell with many improvements and features.

SHIV_DHAGE

What is Bash Shell Scripting?

Bash (Bourne Again Shell) is a Unix shell and command language. Bash scripting involves writing scripts specifically for the Bash shell, leveraging its powerful features and syntax.

Importance of Shell Scripting in Linux for DevOps Culture

Shell scripting is vital in Linux-based DevOps environments due to its ability to

1. **Automate Repetitive Tasks:**
   - **Streamline Processes:** Automates routine tasks such as backups, updates, and monitoring, saving time and reducing manual effort.
   - **Batch Operations:** Handles multiple files or processes simultaneously, enhancing efficiency.
2. **Manage System Configurations:**
   - **Consistent Setup:** Automates the setup and configuration of systems and applications, ensuring uniformity across different environments.
   - **Configuration Changes:** Easily updates and manages configuration files and environment settings.
3. **Facilitate Continuous Integration/Continuous Deployment (CI/CD):**
   - **Pipeline Integration:** Integrates with CI/CD tools to automate build, test, and deployment processes, leading to faster and more reliable releases.
   - **Deployment Scripts:** Manages deployment tasks, including application installation, updates, and rollbacks.
4. **Orchestrate Complex Workflows:**

- **Task Coordination:** Coordinates multiple tasks and services, such as provisioning environments, running tests, and deploying code, into a unified workflow.
- **Scheduled Jobs:** Uses scheduling tools like cron to automate routine tasks and ensure timely execution.

5. **Enhance Productivity and Efficiency:**
   - **Reduce Errors:** Minimizes human errors through automated scripts, leading to more consistent and reliable operations.
   - **Improve Speed:** Speeds up repetitive and complex tasks, boosting overall productivity.

Setting Up Shell Scripting on AWS and MobaXterm

1. Creating a Linux Machine on AWS

1. **Sign In to AWS Console:**
   - Go to AWS Management Console.
   - Sign in with your AWS account credentials.
2. **Launch an EC2 Instance:**
   - Navigate to the EC2 Dashboard.
   - Click on "Launch Instance".
   - Choose an Amazon Machine Image (AMI). For example, select "Amazon Linux 2 AMI".
   - Choose an Instance Type. For beginners, the t2.micro instance type is a good choice.
   - Configure Instance Details as needed.
   - Add Storage if required.
   - Add Tags (optional).
   - Configure Security Group:
     - Add a rule to allow SSH traffic. Set the port to 22 and the source to your IP address or anywhere (0.0.0.0/0) if you want to access it from any location.
   - Review and Launch the instance.
   - Generate a new key pair or use an existing one. Download the key pair (.pem file) and keep it safe.
3. **Connect to Your Instance:**
   - Once the instance is running, select it and click on "Connect".
   - Follow the instructions to connect to the instance via SSH. For example, using a terminal:

```
ssh -i "your-key-pair.pem" ec2-user@your-instance-public-ip
```

2. Connect to EC2 Instance Using MobaXterm

1. **Install MobaXterm:**
   - Download and install MobaXterm on your local machine.
2. **Connect to EC2 Instance:**

SHIV_DHAGE

- Open MobaXterm.
- Click on “Session” in the top left corner.
- Choose “SSH”.
- Enter the public IP address of your EC2 instance.
- Use the username ec2-user.
- In the “Advanced SSH settings” tab, specify the path to your .pem key file.
- Click “OK” to connect.

3. Create and Execute Your First Shell Script

1. **Create a Script File:**
   - Once connected to your EC2 instance, create a new shell script file:

```
touch first_script.sh
```

2. **Open the Script in VI Editor:**
   - Edit the file using the VI editor:

```
vi first_script.sh
```

3. **Edit the Script:**
   - Press i to enter insert mode.
   - Add the following lines to the script:

```
#!/bin/bash
echo "Hello, World!"
```

4. **Save and Exit VI Editor:**
   - Press Esc to exit insert mode.
   - Type :wq and press Enter to save and exit.

5. **Add Execution Permissions:**
   - Make the script executable:

```
chmod +x first_script.sh
```

6. **Execute the Script:**
   - Run the script:

```
./first_script.sh
```

   - You should see the output:

```
Hello, World!
```

SHIV_DHAGE

By following these steps, you have successfully set up a Linux machine on AWS, connected to it using MobaXterm, created a shell script, and executed it. This setup provides a foundational environment for practicing and developing more complex shell scripts.

Variables, Control Flow Statements, Case Statements, Loops, and Functions

1. Variables in Shell Scripting

Definition: Variables are used to store data that can be referenced and manipulated within a script.

Syntax:

```
variable_name=value
```

Examples:

1. Assigning and Accessing Variables:

```
# Assign a value to a variable
 NAME="John"

# Access the value of a variable
echo "Hello, $NAME"
```

Output:

```
Hello, John
```

2.Using Variables in Arithmetic Operations:

```
# Assign values to variables
NUM1=5
NUM2=10

# Perform arithmetic operations
SUM=$((NUM1 + NUM2))
echo "Sum: $SUM"
```

Output:

```
makefile
Copy code
```

```
Sum: 15
```

2. Control Flow Statements

Definition: Control flow statements allow the execution of different parts of a script based on conditions.

Types:

- if statement
- if-else statement
- if-elif-else statement

Examples:

Simple if Statement:

```
#!/bin/bash
NUM=10
if [ $NUM -gt 5 ]; then
   echo "Number is greater than 5"
fi
```

Output:

```
Number is greater than 5
```

if-else Statement:

```
#!/bin/bash
NUM=3
if [ $NUM -gt 5 ]; then
   echo "Number is greater than 5"
else
   echo "Number is not greater than 5"
fi
```

Output:

```
Number is not greater than 5
```

if-elif-else Statement:

```
#!/bin/bash
NUM=7
if [ $NUM -gt 10 ]; then
   echo "Number is greater than 10"
elif [ $NUM -gt 5 ]; then
```

```
    echo "Number is greater than 5 but less than or equal
to 10"
 else
    echo "Number is 5 or less"
 fi
```

Output:

```
Vbnet
Number is greater than 5 but less than or equal to 10
```

3. Case Statements

Definition: Case statements are used for pattern matching and executing commands based on specific patterns.

Syntax:

```
Case expression in
   pattern1)
       commands ;;
   pattern2)
       commands ;;
   *)
       default commands ;;
esac
```

Example:

```
#!/bin/bash
DAY="Monday"

case $DAY in
   "Monday")
       echo "Start of the work week" ;;
   "Friday")
       echo "End of the work week" ;;
   "Saturday" | "Sunday")
       echo "Weekend!" ;;
   *)
       echo "Midweek day" ;;
esac
```

Output:

```
Start of the work week
```

4. Loops in Shell Scripting

Definition: Loops are used to execute a block of commands repeatedly.

Types:

- for loop
- while loop
- until loop

Examples:

- **for Loop:**

```
#!/bin/bash
for i in 1 2 3 4 5; do
   echo "Number: $i"
done
```

Output:

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

while Loop:

```
#!/bin/bash
COUNTER=1
while [ $COUNTER -le 5 ]; do
   echo "Counter: $COUNTER"
   COUNTER=$((COUNTER + 1))
done
```

Output:

```
Counter: 1
Counter: 2
Counter: 3
Counter: 4
Counter: 5
```

until Loop:

```
#!/bin/bash
COUNTER=1
```

```
until [ $COUNTER -gt 5 ]; do
    echo "Counter: $COUNTER"
    COUNTER=$((COUNTER + 1))
done
```

Output:

```
Counter: 1
Counter: 2
Counter: 3
Counter: 4
Counter: 5
```

5. Functions in Shell Scripting

Definition: Functions are blocks of code that perform a specific task and can be called multiple times within a script.

Syntax:

```
function_name() {
    commands
}
```

Examples:

Simple Function:

```
#!/bin/bash
greet() {
    echo "Hello, $1"
}

greet "Alice"
greet "Bob"
```

Output:

```
Hello, Alice
Hello, Bob
```

Function with Return Value:

```
#!/bin/bash
add() {
    SUM=$(( $1 + $2 ))
    return $SUM
}
```

```
add 3 4
echo "Sum: $?"
```

Output:

```
Sum: 7
```

These examples cover the basics and essential aspects of shell scripting, providing a strong foundation for more advanced scripting and automation tasks in a DevOps environment.

Command Line Arguments in Shell Scripting

Definition: Command line arguments are parameters passed to a shell script at the time of its execution. They allow users to provide input to the script without hardcoding values.

Syntax:

- $0: The name of the script.
- $1, $2, ..., $N: The first, second, ..., Nth command line arguments.
- $#: The number of arguments supplied to the script.
- $@: All the arguments supplied to the script.
- $*: All the arguments supplied to the script as a single string.
- "$@": Each argument is a quoted string, treated as separate.
- "$*": All arguments are quoted as a single string.

SHIV DHAGE

Example Script:

Create a script called args_example.sh:

```
#!/bin/bash

echo "Script Name: $0"
echo "First Argument: $1"
echo "Second Argument: $2"
echo "All Arguments as a Single String: $*"
echo "All Arguments as Separate Strings: $@"
echo "Number of Arguments: $#"

# Loop through all arguments
for arg in "$@"; do
   echo "Argument: $arg"
done
```

Usage and Output:

1. **Save the Script:**

```
vi args_example.sh
```

Paste the script content and save it by pressing Esc, typing :wq, and pressing Enter.

2. **Make the Script Executable:**

```
chmod +x args_example.sh
```

3. **Execute the Script with Arguments:**

```
./args_example.sh arg1 arg2 arg3
```

4. **Expected Output:**

```
Script Name: ./args_example.sh
First Argument: arg1
Second Argument: arg2
All Arguments as a Single String: arg1 arg2 arg3
All Arguments as Separate Strings: arg1 arg2 arg3
Number of Arguments: 3
Argument: arg1
Argument: arg2
Argument: arg3
```

Detailed Explanation:

- **$0:** Represents the name of the script being executed. In this example, it will be ./args_example.sh.
- **$1, $2:** Represent the first and second arguments passed to the script. If you run ./args_example.sh arg1 arg2 arg3, $1 will be arg1 and $2 will be arg2.
- **$*:** Represents all the arguments passed to the script as a single string.
- **$@:** Represents all the arguments passed to the script as separate strings.
- **$#:** Represents the total number of arguments passed to the script.
- **for arg in "$@"; do ... done:** Loops through each argument passed to the script and prints it.

Command line arguments provide a powerful way to make scripts dynamic and flexible, allowing users to supply different inputs at runtime. This is especially useful in automation and batch processing tasks.

Commonly Used Shell Scripting Commands for DevOps

1. Basic File and Directory Commands

- **Create a new file:**

```
touch filename
```

- **List files and directories:**

```
ls
```

- **Change directory:**

```
cd /path/to/directory
```

- **Create a new directory:**

```
mkdir directoryname
```

- **Remove a file:**

```
rm filename
```

- **Remove a directory and its contents:**

```
rm -r directoryname
```

- **Copy a file:**

```
cp sourcefile destinationfile
```

- **Move/Rename a file:**

```
mv sourcefile destinationfile
```

- **Display the contents of a file:**

```
cat filename
```

- **Display the first 10 lines of a file:**

```
head filename
```

- **Display the last 10 lines of a file:**

```
tail filename
```

SHIV_DHAGE

2. Text Processing Commands

- **Search for a pattern in a file:**

```
grep "pattern" filename
```

- **Replace text in a file using sed:**

```
sed 's/oldtext/newtext/g' filename
```

- **Sort lines in a file:**

```
sort filename
```

- **Count words, lines, and characters in a file:**

```
wc filename
```

3. System Monitoring and Management Commands

- **Check disk space usage:**

```
df -h
```

- **Check memory usage:**

```
free -h
```

- **Display system uptime:**

```
uptime
```

- **Display running processes:**

```
ps aux
```

- **Display system information:**

```
uname -a
```

- **Monitor real-time system activity:**

```
top
```

SHIV_DHAGE

4. Network Commands

- **Check network connectivity:**

```
ping hostname_or_ip
```

- **Display network interfaces and IP addresses:**

```
ifconfig
```

- **Display network routing table:**

```
netstat -r
```

- **Transfer files over SSH:**

```
scp localfile user@remotehost:/path/to/destination
```

5. Shell Scripting Basics

- **Define a variable:**

```
VARIABLE_NAME="value"
```

- **Access a variable:**

```
echo $VARIABLE_NAME
```

- **Simple if statement:**

```
if [ condition ]; then
    # commands
fi
```

- **if-else statement:**

```
if [ condition ]; then
    # commands
else
    # commands
fi
```

- **for loop:**

```
for variable in list; do
    # commands
done
```

SHIV_DHAGE

- **while loop:**

```
while [ condition ]; do
    # commands
done
```

6. Advanced Shell Scripting

- **Define a function:**

```
function_name() {
    # commands
}
```

- **Call a function:**

```
function_name
```

- **Pass arguments to a script:**

```
./script.sh arg1 arg2
```

- **Access script arguments:**

```
echo "First argument: $1"
echo "Second argument: $2"
```

- **Basic case statement:**

```
Case "$variable" in
    pattern1)
        # commands ;;
    pattern2)
        # commands ;;
    *)
        # default commands ;;
esac
```

SHIV DHAGE

7. Example Scripts

- **Print "Hello, World!":**

```
#!/bin/bash
echo "Hello, World!"
```

- **Check if a file exists:**

```
#!/bin/bash
if [ -f "filename" ]; then
   echo "File exists."
else
   echo "File does not exist."
fi
```

- **Loop through a list of numbers:**

```
#!/bin/bash
for i in 1 2 3 4 5; do
   echo "Number: $i"
done
```

- **Find the greatest number between two inputs:**

```
#!/bin/bash
echo "Enter first number:"
read a
echo "Enter second number:"
read b
if [ $a -gt $b ]; then
   echo "$a is greater than $b"
else
   echo "$b is greater than $a"
fi
```

These commands and examples provide a solid foundation for daily DevOps tasks and shell scripting. By mastering these, you'll be well-equipped to handle a wide range of automation and management tasks in a DevOps environment.

Example 1: Finding the Greater of Two Numbers

Script Name: find_greater.sh

Step-by-Step Instructions:

1. **Create the Script File:**

   Open a terminal and use a text editor (e.g., vi or nano) to create the script file.

```
vi find_greater.sh
```

2. **Add the Script Content:**

   Paste the following content into the file:

```
#!/bin/bash

# This script finds the greater of two numbers

# Check if exactly two arguments are provided
if [ $# -ne 2 ]; then
   echo "Usage: $0 number1 number2"
   exit 1
fi

# Assign arguments to variables
NUM1=$1
NUM2=$2

# Compare the two numbers
if [ $NUM1 -gt $NUM2 ]; then
   echo "$NUM1 is greater than $NUM2"
elif [ $NUM1 -lt $NUM2 ]; then
   echo "$NUM2 is greater than $NUM1"
else
   echo "$NUM1 and $NUM2 are equal"
fi
```

   Save and exit the editor (for vi, press Esc, type :wq, and press Enter).

3. **Make the Script Executable:**

```
chmod +x find_greater.sh
```

4. **Execute the Script:**

```
./find_greater.sh 10 20
```

   Output:

```
20 is greater than 10
```

Try different inputs to see how it works.

Example 2: Simple Calculator (Addition of Two Numbers)

Script Name: simple_calculator.sh

Step-by-Step Instructions:

1. **Create the Script File:**

   Open a terminal and create the script file.

```
vi simple_calculator.sh
```

2. **Add the Script Content:**

   Paste the following content into the file:

```
#!/bin/bash

# This script performs addition of two numbers

# Check if exactly two arguments are provided
if [ $# -ne 2 ]; then
   echo "Usage: $0 number1 number2"
   exit 1
fi

# Assign arguments to variables
NUM1=$1
NUM2=$2

# Perform addition
SUM=$((NUM1 + NUM2))

# Display the result
echo "The sum of $NUM1 and $NUM2 is $SUM"
```

   Save and exit the editor.

3. **Make the Script Executable:**

DHACE

```
chmod +x simple_calculator.sh
```

4. **Execute the Script:**

```
./simple_calculator.sh 5 15
```

Output:

```
The sum of 5 and 15 is 20
```

These examples illustrate how to create, format, and execute shell scripts for common tasks. The process involves creating a file, adding script content, making the file executable, and then running the script with appropriate arguments.

SHIV_DHAGE